Aether Benchmark Analysis Note

Scope

All runs used the shared-guide path for Aether:

- `--shared-guide-batch-size 4`
- Aether was benchmarked with both guide variants:
- `Docs/aether_for_llms_and_others.md`
- `Docs/aether_for_llms_with_small_contexts.md`
- Python baseline was run per task with no guide payload

Reports:

- `/tmp/aether_bench_gemini25flash_batch4_python.json`
- `/tmp/aether_bench_gemini25flashlite_batch4_python.json`
- `/tmp/aether_bench_qwen35_2b_batch4_python.json`

1. Accuracy

Model	Guide	Aether Generated	Aether Run	Aether Exact	Python Generated	Python Run	Python Exact
`gemini-2.5-flash`	full	25/25	21/25	20/25	25/25	14/25	13/25
`gemini-2.5-flash`	small	25/25	22/25	19/25	25/25	15/25	12/25
`gemini-2.5-flash-lite`	full	25/25	17/25	16/25	25/25	18/25	12/25
`gemini-2.5-flash-lite`	small	25/25	20/25	19/25	25/25	18/25	13/25
`qwen3.5-2b-mlx`	full	25/25	16/25	6/25	25/25	11/25	6/25
`qwen3.5-2b-mlx`	small	22/25	9/25	6/25	25/25	11/25	6/25

Reading that table:

- Aether beat Python on exact-match accuracy for both Gemini models in every tested guide configuration.
- `gemini-2.5-flash-lite` improved materially with the small guide: `16/25` exact to `19/25`.
- `gemini-2.5-flash` stayed strong with both guides; full guide still wins slightly on exactness.

- The 2B local model is the outlier: Aether and Python tie on exactness, and the small guide hurts Aether generation stability.

2. Workflow Token Cost

These are total provider-reported workflow tokens across the full 25-task run.

Model	Guide	Aether Total Tokens	Python Total Tokens	Aether / Python
`gemini-2.5-flash`	full	189,689	42,988	4.41x
`gemini-2.5-flash`	small	81,065	44,297	1.83x
`gemini-2.5-flash-lite`	full	98,126	11,613	8.45x
`gemini-2.5-flash-lite`	small	43,842	11,685	3.75x
`qwen3.5-2b-mlx`	full	280,146	13,685	20.47x
`qwen3.5-2b-mlx`	small	58,452	11,292	5.18x

Reading that table:

- The small guide is doing real work. It cuts Aether workflow token cost sharply.
- The strongest current Aether workflow configuration here is `gemini-2.5-flash-lite` + small guide: `19/25` exact and `43,842` total tokens.
- Even after batching, Python is still cheaper at workflow level.
- The remaining Aether overhead is mostly guide tax, not necessarily final program size.

3. Final Answer Size

This uses average approximate tokens for exact-match final answers only.

Model	Guide	Aether Exact Final Avg	Python Exact Final Avg
`gemini-2.5-flash`	full	106.35	69.62
`gemini-2.5-flash`	small	100.84	105.92
`gemini-2.5-flash-lite`	full	70.19	112.08
`gemini-2.5-flash-lite`	small	134.42	136.08
`qwen3.5-2b-mlx`	full	89.67	48.67
`qwen3.5-2b-mlx`	small	80.67	42.00

Reading that table:

- This is the more favorable lens for Aether.
- For `gemini-2.5-flash` with the small guide, Aether and Python final answer sizes are essentially at parity, with Aether slightly smaller.
- For `gemini-2.5-flash-lite`, Aether is smaller than Python with the full guide, and near parity with the small guide.
- So the conclusion that Aether is always bigger is not supported by final-answer size data.
- The real issue remains prompt overhead, especially for weaker models.

4. What the Runs Suggest

Observation	Likely Meaning
Gemini models outperform Python on exactness	Aether's constraints are helping stronger models stay on target
Small guide greatly reduces token cost	The shared-guide batching path is worthwhile
Small guide improves `flash-lite` but slightly hurts `flash`	Different models want different amounts of scaffolding
2B model ties Python but at much higher Aether token cost	Tiny models still struggle to internalize Aether reliably
Repeated `AETH-RUNTIME-TOON-GET-INDEX-ARRAY` failures	Real language/runtime pain point worth fixing or guarding better
Repeated `AETH-REWRITE-LET-INFER` failures	Inference rules are still a recurring friction surface

5. Bottom Line

Question	Current Answer
Is Aether more accurate than Python for capable models?	Yes, in these runs
Is Aether cheaper than Python at workflow level?	Not yet
Is Aether competitive at final answer size?	Often yes
Does batching help?	Yes
Is the small guide useful?	Yes, especially for token cost and for `flash-lite`
Are there still compiler/doc targets clearly visible?	Yes: TOON indexing and `let` inference remain high-value targets

The strongest current practical configuration from this set looks like `gemini-2.5-flash-lite` with the small guide: high exactness, much lower Aether token cost than the full guide, and still a clear accuracy edge over Python.